ViandasSemana

Diseño técnico

DFD, casos de uso, DER, arquitectura y decisiones técnicas

Segundo Enriquez Gatti · Roman Oberlander
6.° 1.ª — E.T. N.° 21 — Programación sobre Redes — 2026

Índice

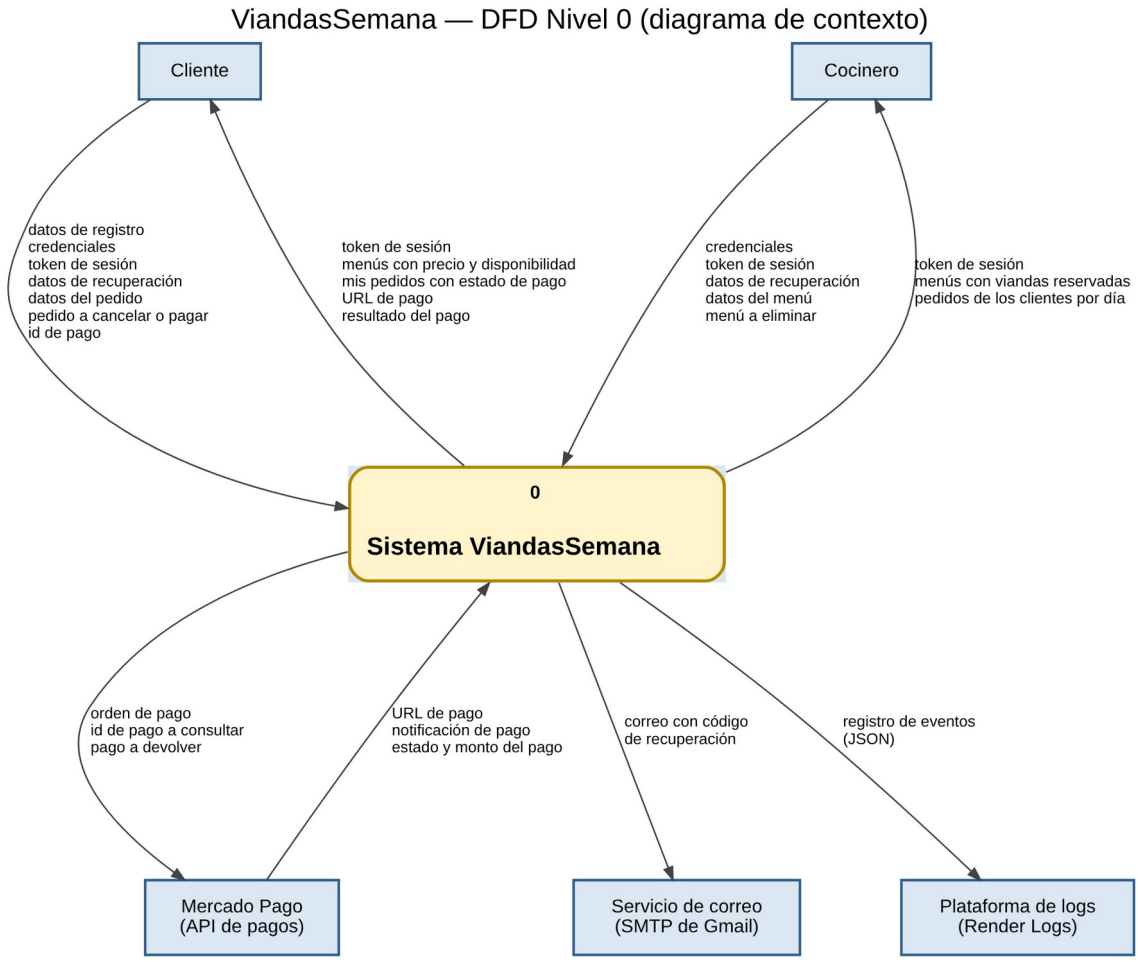
1. Diagramas de flujo de datos 3
1.1 Nivel 0 — Diagrama de contexto 3
1.2 Nivel 1 5
1.3 Nivel 2 6
2. Casos de uso 11
2.1 Actores 11
2.2 Catálogo 12
2.3 Relaciones include y extend 12
3. Modelo de datos (DER) 13
3.1 Cardinalidades 13
3.2 Diccionario de datos 14
4. Arquitectura y despliegue 16
4.1 Componentes 16
4.2 Comunicaciones 17
4.3 Secretos 17
4.4 Del push a producción 17
4.5 Ambiente local y de producción 18
5. Decisiones técnicas 19
Declaración de uso de inteligencia artificial 20

1. Diagramas de flujo de datos

Notación **Gane-Sarson**: entidades externas en rectángulos azules, procesos en rectángulos redondeados amarillos con su número y almacenes de datos en rectángulos abiertos verdes con su código (D1 a D5). Cada flujo lleva el **dato** que viaja, no la acción que lo origina.

Todas las peticiones de Cliente y Cocinero (salvo registro, inicio de sesión y recuperación) llevan el token de sesión; por eso "token de sesión" entra al proceso 1, que entrega "identidad y rol" a los demás procesos.

1.1 Nivel 0 — Diagrama de contexto

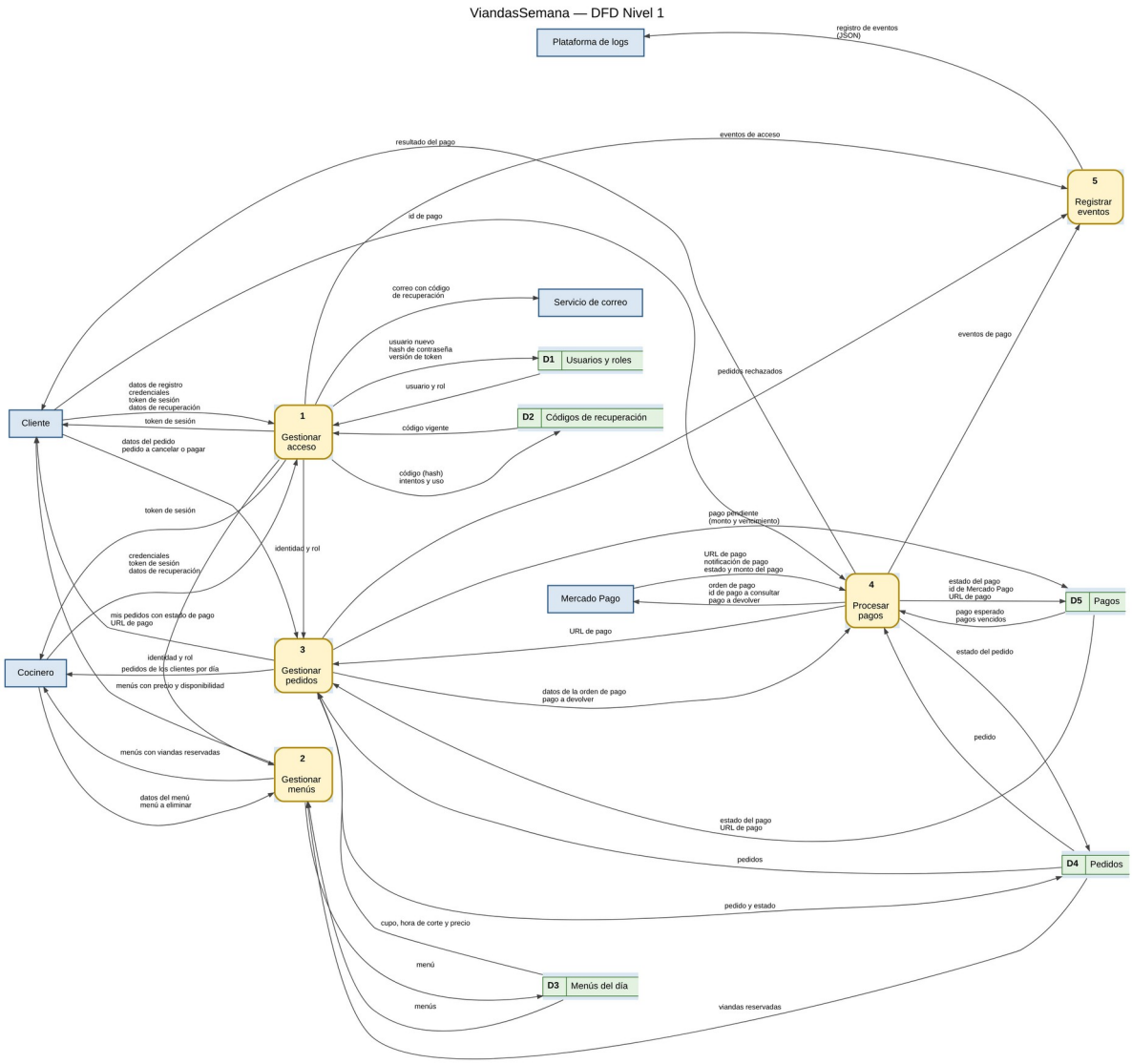


Archivo editable: diagramas/dfd-0-contexto.drawio.

Entidad externa	Flujos de entrada al sistema	Flujos de salida del sistema
Cliente	datos de registro · credenciales · token de sesión · datos de recuperación · datos del pedido · pedido a cancelar o pagar · id de pago	token de sesión · menús con precio y disponibilidad · mis pedidos con estado de pago · URL de pago · resultado del pago
Cocinero	credenciales · token de sesión · datos de recuperación · datos del menú · menú a eliminar	token de sesión · menús con viandas reservadas · pedidos de los clientes por día
Mercado Pago	URL de pago · notificación de pago · estado y monto del pago	orden de pago · id de pago a consultar · pago a devolver
Servicio de correo	—	correo con código de recuperación

Entidad externa	Flujos de entrada al sistema	Flujos de salida del sistema
Plataforma de logs	—	registro de eventos (JSON)

1.2 Nivel 1



Archivo editable: diagramas/dfd-1.drawio.

N.º	Proceso	Módulo	Almacenes
1	Gestionar acceso	Módulo 1 — Autenticación	D1 Usuarios y roles, D2 Códigos de recuperación
2	Gestionar menús	Módulo 2 — Menús	D3 Menús del día, D4 Pedidos (lectura)
3	Gestionar pedidos	Módulo 3 — Pedidos	D3, D4, D5
4	Procesar pagos	Módulo 4 — Pagos	D4, D5
5	Registrar eventos	Módulo 5 — Plataforma	— (los eventos van a la plataforma de logs)

Almacén	Tabla(s) de la base
D1 Usuarios y roles	Usuario, Rol
D2 Códigos de recuperación	CodigoRecuperacion
D3 Menús del día	MenuDia

N.º	Proceso	Módulo	Almacenes
D4 Pedidos		Pedido	
D5 Pagos		Pago	

Consistencia entre el Nivel 0 y el Nivel 1

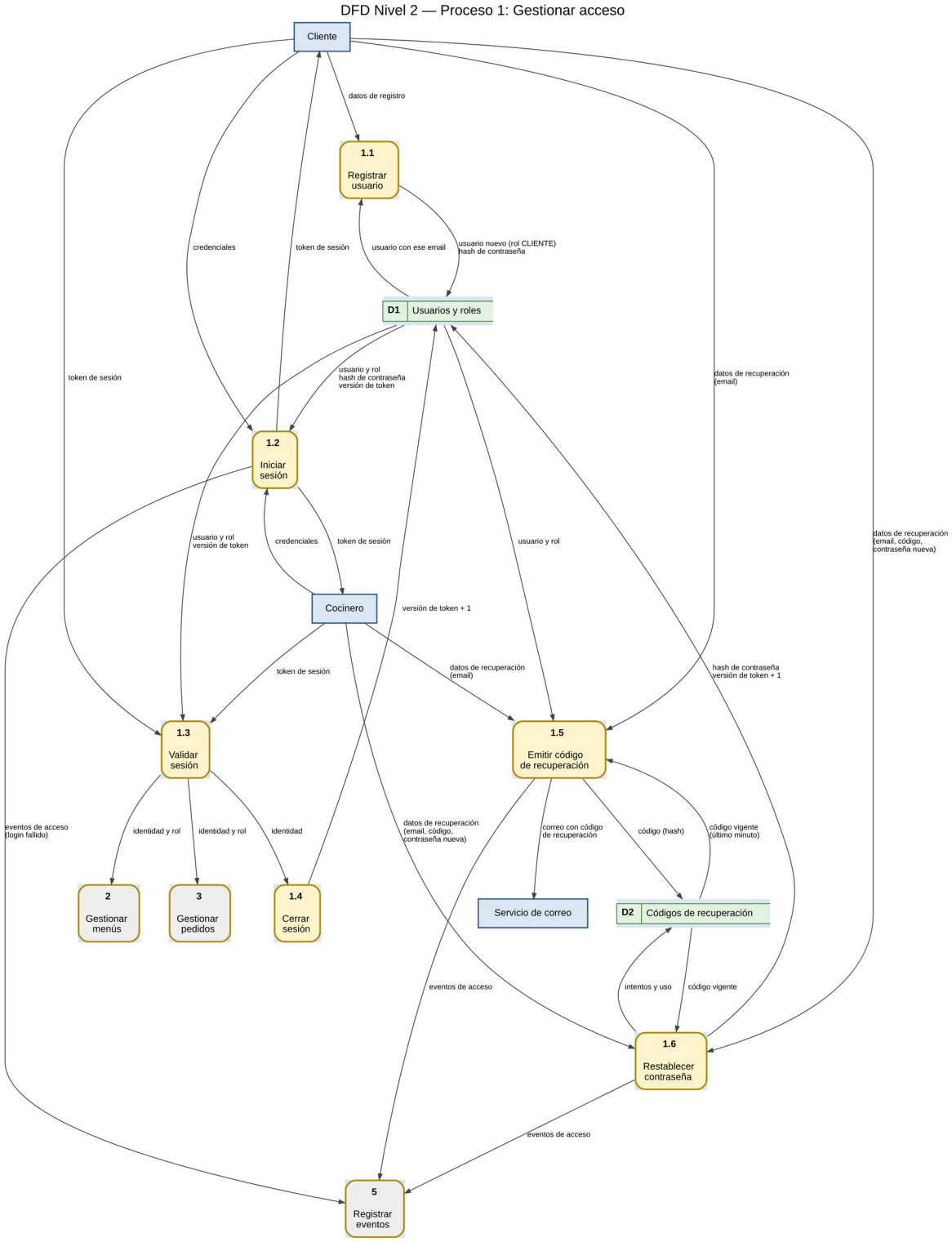
Cada flujo del diagrama de contexto aparece en el Nivel 1 con el mismo nombre, conectado al proceso que lo produce o lo consume. No hay en el Nivel 1 flujos con entidades externas que no estén en el Nivel 0.

Flujo del Nivel 0	Sentido	Proceso
datos de registro	Cliente → sistema	1
credenciales · token de sesión · datos de recuperación	Cliente, Cocinero → sistema	1
token de sesión	sistema → Cliente, Cocinero	1
correo con código de recuperación	sistema → Servicio de correo	1
datos del menú · menú a eliminar	Cocinero → sistema	2
menús con viandas reservadas	sistema → Cocinero	2
menús con precio y disponibilidad	sistema → Cliente	2
datos del pedido · pedido a cancelar o pagar	Cliente → sistema	3
mis pedidos con estado de pago · URL de pago	sistema → Cliente	3
pedidos de los clientes por día	sistema → Cocinero	3
id de pago	Cliente → sistema	4
resultado del pago	sistema → Cliente	4
orden de pago · id de pago a consultar · pago a devolver	sistema → Mercado Pago	4
URL de pago · notificación de pago · estado y monto del pago	Mercado Pago → sistema	4
registro de eventos (JSON)	sistema → Plataforma de logs	5

1.3 Nivel 2

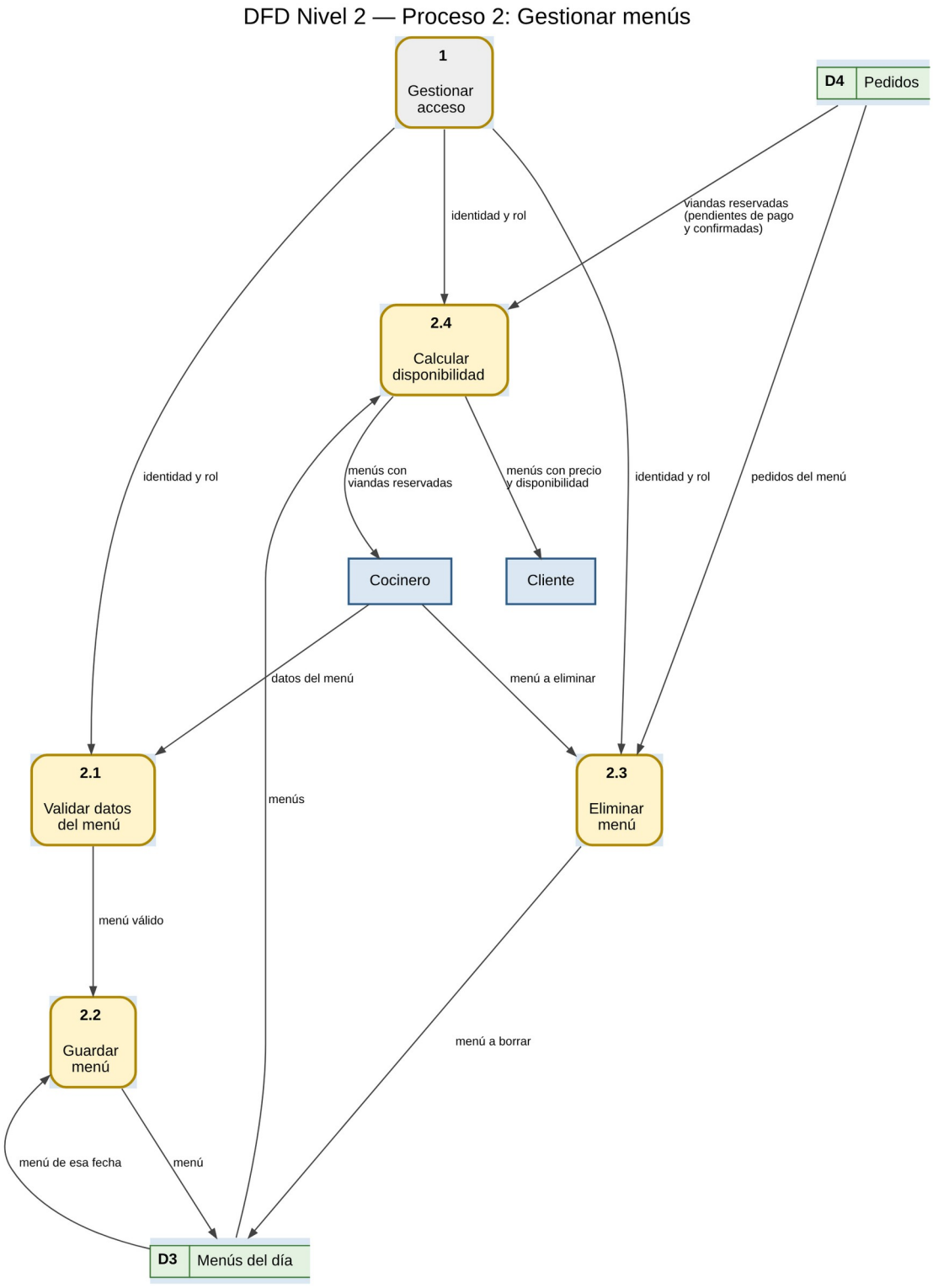
Se descomponen los cuatro procesos que manejan datos del negocio. La numeración es jerárquica y cada diagrama usa sólo los flujos que su proceso tiene en el Nivel 1. La tabla de cada subproceso está en el documento de su módulo.

Proceso 1 — Gestionar acceso



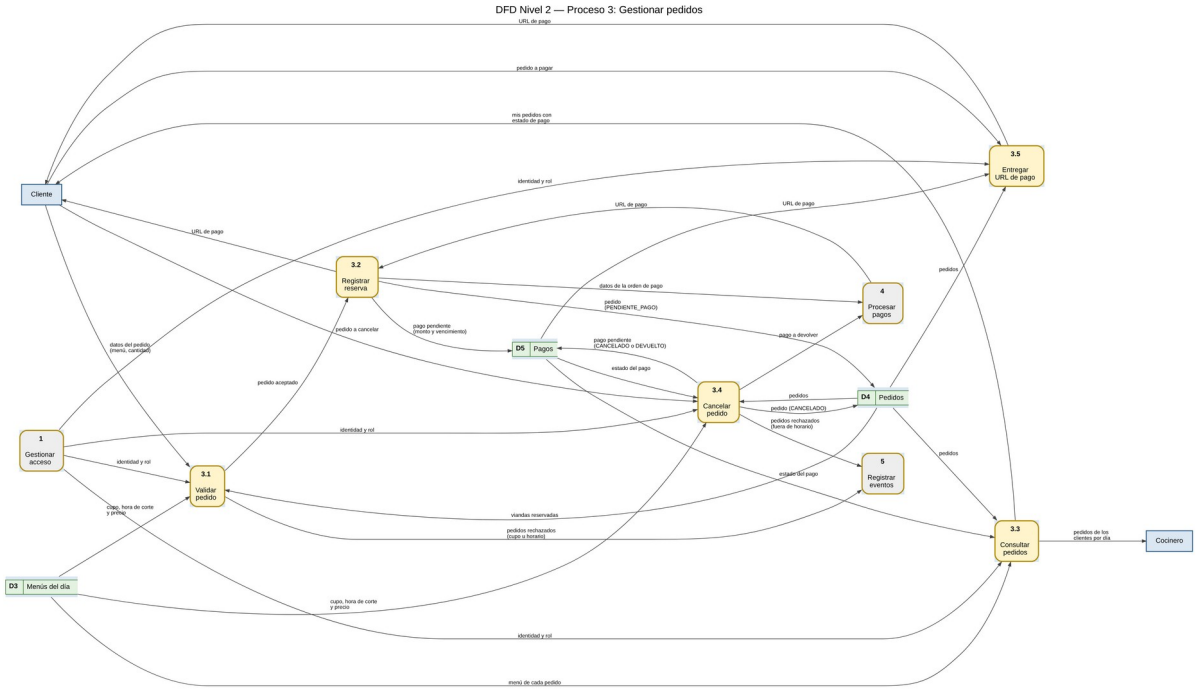
Archivo editable: diagramas/dfd-2-acceso.drawio.

Proceso 2 — Gestionar menús



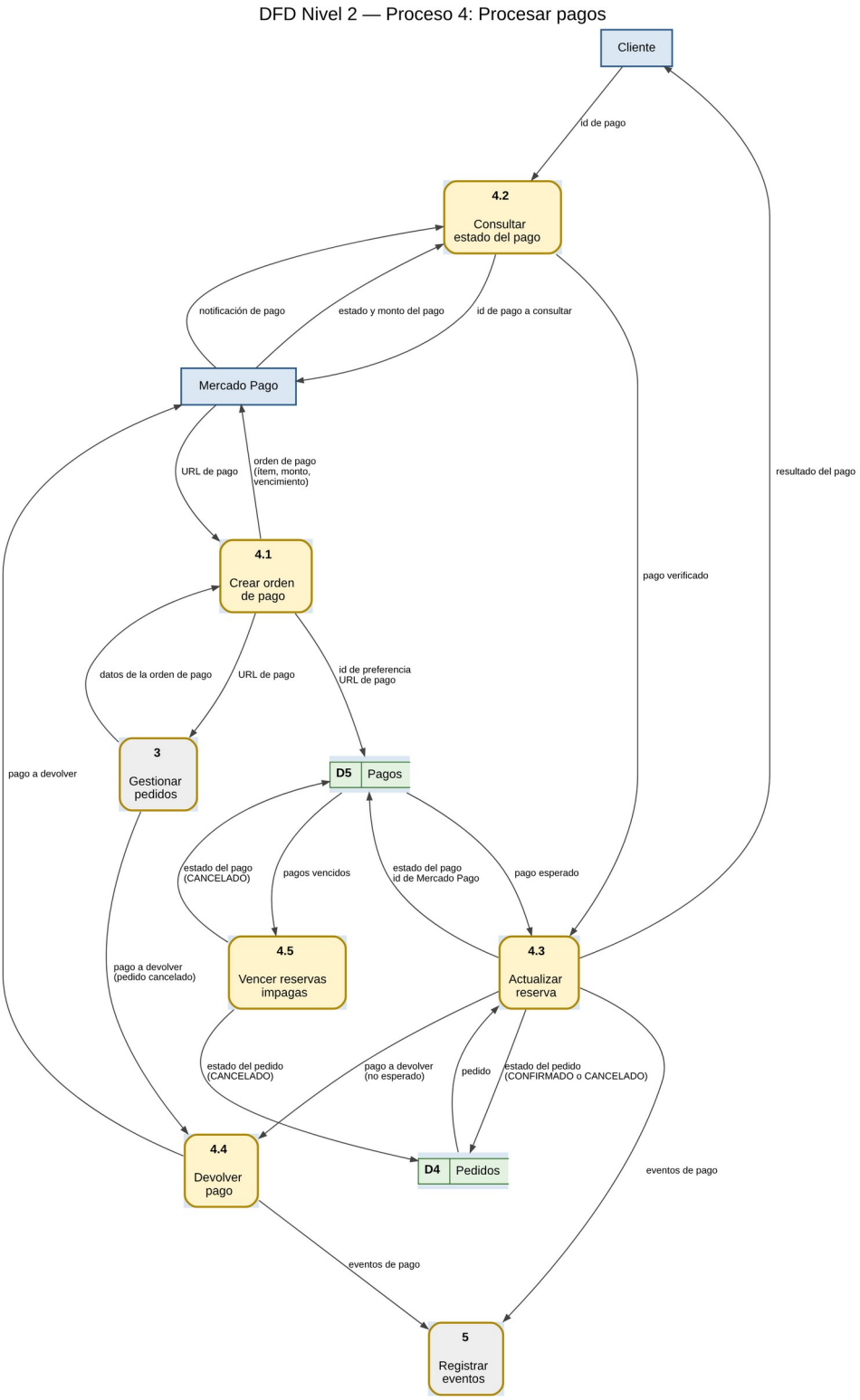
Archivo editable: diagramas/dfd-2-menus.drawio.

Proceso 3 — Gestionar pedidos



Archivo editable: diagramas/dfd-2-pedidos.drawio.

Proceso 4 — Procesar pagos



Archivo editable: diagramas/dfd-2-pagos.drawio.

2. Casos de uso

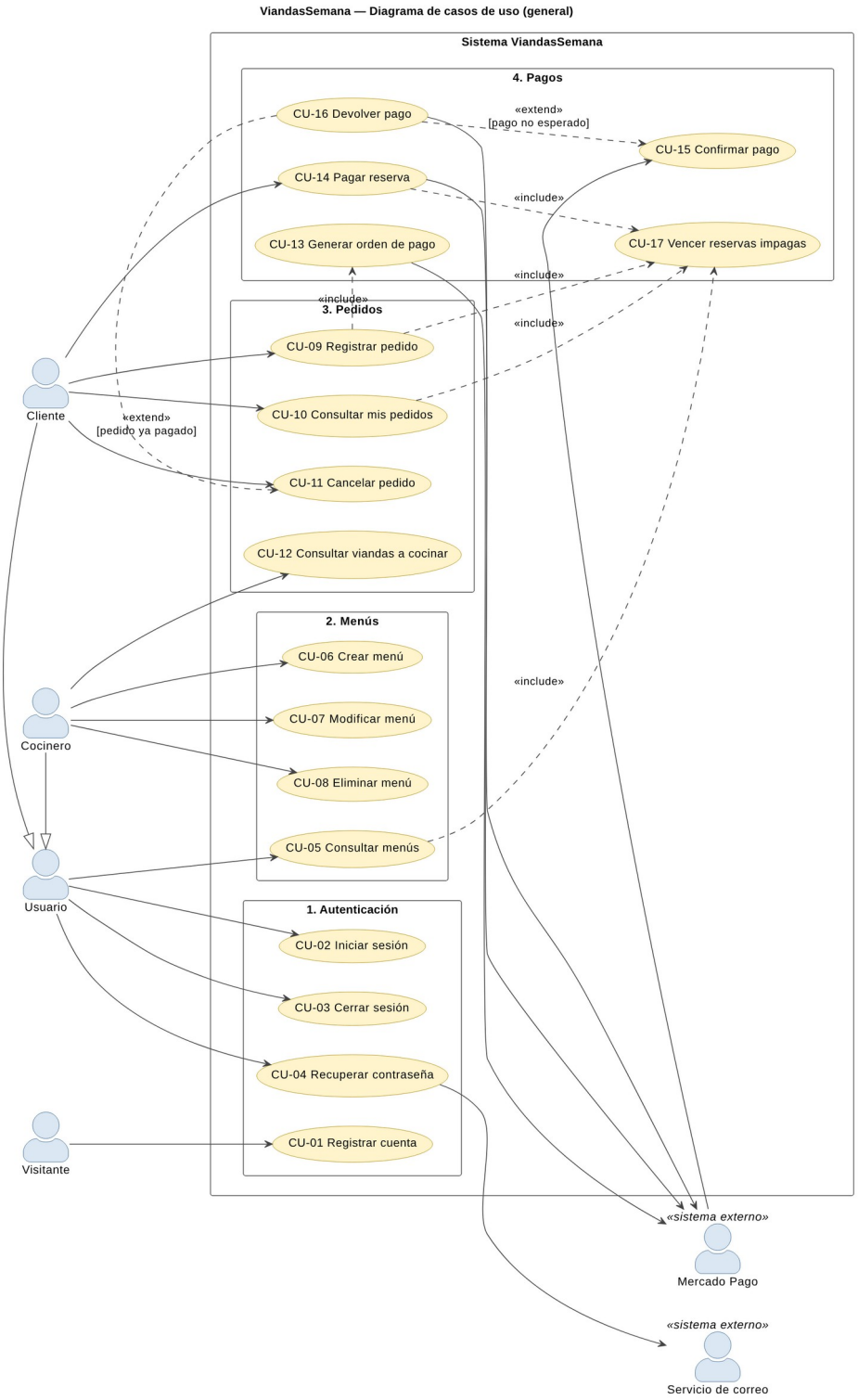


Diagrama general de casos de uso (archivo editable: diagramas/cu-general.drawio).

2.1 Actores

Actor	Tipo	Participa en
Visitante	humano	CU-01
Usuario (generalización de Cliente y Cocinero)	humano	CU-02, CU-03, CU-04, CU-05

Actor	Tipo	Participa en
Cliente	humano	CU-09, CU-10, CU-11, CU-14 (y los de Usuario)
Cocinero	humano	CU-06, CU-07, CU-08, CU-12 (y los de Usuario)
Mercado Pago	sistema externo	CU-13, CU-14, CU-15 (avisa los pagos por webhook), CU-16
Servicio de correo	sistema externo	CU-04

2.2 Catálogo

Módulo	Caso de uso	Actor principal
1 Autenticación	CU-01 Registrar cuenta · CU-02 Iniciar sesión · CU-03 Cerrar sesión · CU-04 Recuperar contraseña	Visitante / Usuario
2 Menús	CU-05 Consultar menús · CU-06 Crear menú · CU-07 Modificar menú · CU-08 Eliminar menú	Usuario / Cocinero
3 Pedidos	**CU-09 Registrar pedido** · CU-10 Consultar mis pedidos · **CU-11 Cancelar pedido** · CU-12 Consultar viandas a cocinar	Cliente / Cocinero
4 Pagos	CU-13 Generar orden de pago · CU-14 Pagar reserva · **CU-15 Confirmar pago** · CU-16 Devolver pago · CU-17 Vencer reservas impagas	Cliente / Mercado Pago

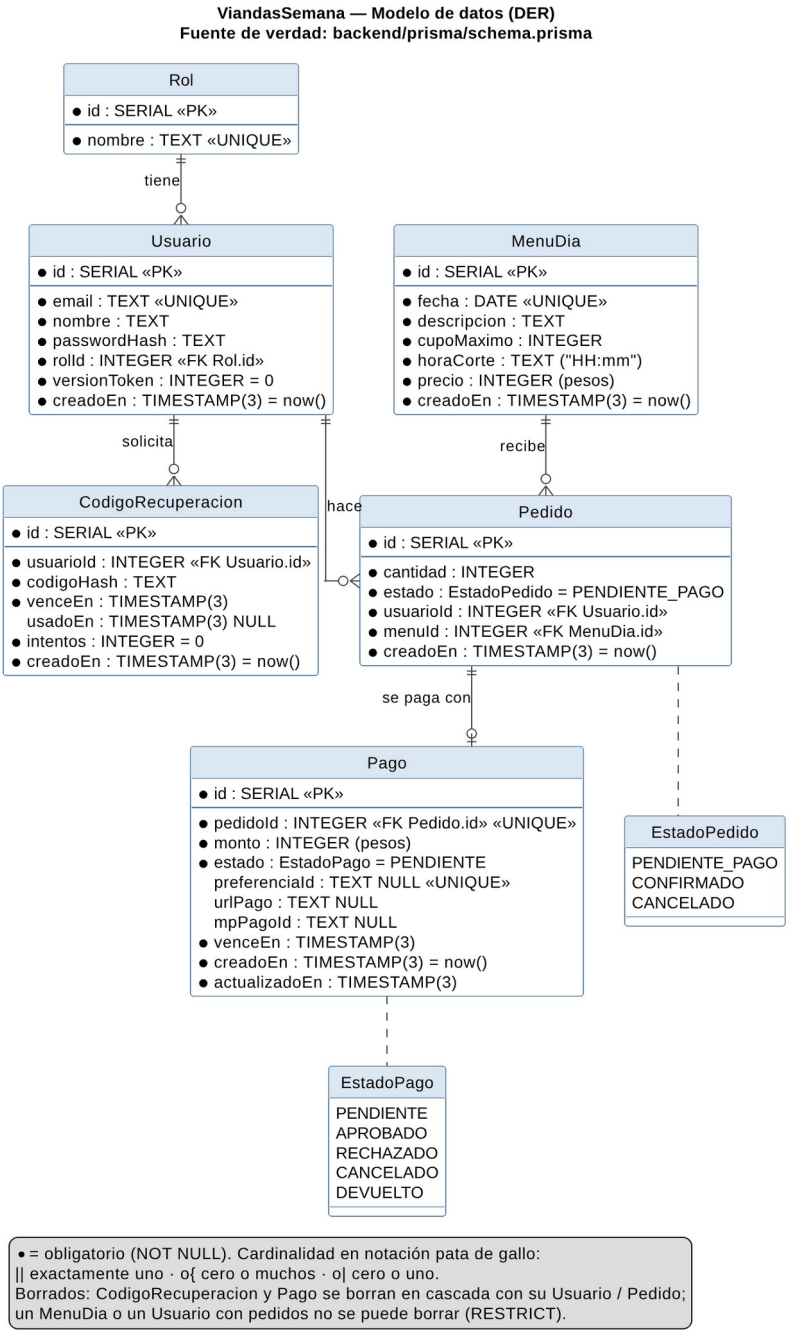
En negrita, los tres casos de uso principales. La plantilla completa de **cada** caso de uso está en el documento de su módulo.

2.3 Relaciones include y extend

Relación	Por qué
CU-09 «include» CU-13 Generar orden de pago	Toda reserva genera siempre su orden de pago: sin ella la reserva se cancela.
CU-05, CU-09, CU-10, CU-14 «include» CU-17 Vencer reservas impagas	Se ejecuta siempre antes de mostrar o contar el cupo.
CU-16 Devolver pago «extend» CU-11 Cancelar pedido	Sólo si el pedido cancelado ya estaba pagado.
CU-16 Devolver pago «extend» CU-15 Confirmar pago	Sólo si llega un pago aprobado no esperado.
Cliente y Cocinero son un Usuario (generalización)	Los dos inician y cierran sesión, recuperan la contraseña y consultan menús.

3. Modelo de datos (DER)

La fuente de verdad es `backend/prisma/schema.prisma`; las tablas se crean con las migraciones de `backend/prisma/migrations/`. El DER refleja exactamente lo implementado en PostgreSQL.



Archivo editable: diagramas/der.drawio.

3.1 Cardinalidades

Relación	Cardinalidad	Clave foránea	Al borrar el padre
Rol — Usuario	1 : N	Usuario.rolId → Rol.id	RESTRICT
Usuario — CodigoRecuperacion	1 : N	CodigoRecuperacion.usuarioId → Usuario.id	CASCADE

Relación	Cardinalidad	Clave foránea	Al borrar el padre
Usuario — Pedido	1 : N	Pedido.usuarioId → Usuario.id	RESTRICT
MenuDia — Pedido	1 : N	Pedido.menuId → MenuDia.id	RESTRICT
Pedido — Pago	1 : 0..1	Pago.pedidoId → Pedido.id (UNIQUE)	CASCADE

3.2 Diccionario de datos

Rol

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
nombre	TEXT	NOT NULL, UNIQUE	COCINERO o CLIENTE (los inserta la migración inicial)

Usuario

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
email	TEXT	NOT NULL, UNIQUE	Se guarda en minúsculas
nombre	TEXT	NOT NULL	
passwordHash	TEXT	NOT NULL	Hash bcrypt (incluye la sal). Nunca la contraseña
rolId	INTEGER	NOT NULL, FK → Rol.id	
versionToken	INTEGER	NOT NULL, DEFAULT 0	Sube en cada logout y al cambiar la contraseña
creadoEn	TIMESTAMP(3)	NOT NULL, DEFAULT now	

CodigoRecuperacion

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
usuarioId	INTEGER	NOT NULL, FK → Usuario.id, índice	
codigoHash	TEXT	NOT NULL	HMAC-SHA256 del código de 6 dígitos. Nunca el código
venceEn	TIMESTAMP(3)	NOT NULL	15 minutos después de creado
usadoEn	TIMESTAMP(3)	NULL	Al usarlo, reemplazarlo o bloquearlo
intentos	INTEGER	NOT NULL, DEFAULT 0	Intentos fallidos (máximo 5)
creadoEn	TIMESTAMP(3)	NOT NULL, DEFAULT now	Límite de un código por minuto

MenuDia

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
fecha	DATE	NOT NULL, UNIQUE	Un menú por día
descripcion	TEXT	NOT NULL	
cupoMaximo	INTEGER	NOT NULL	Viandas que se pueden cocinar ese día (≥ 1)
horaCorte	TEXT	NOT NULL	"HH:mm", hora de Argentina
precio	INTEGER	NOT NULL	Pesos por vianda (0 en menús anteriores a los pagos)
creadoEn	TIMESTAMP(3)	NOT NULL, DEFAULT now	

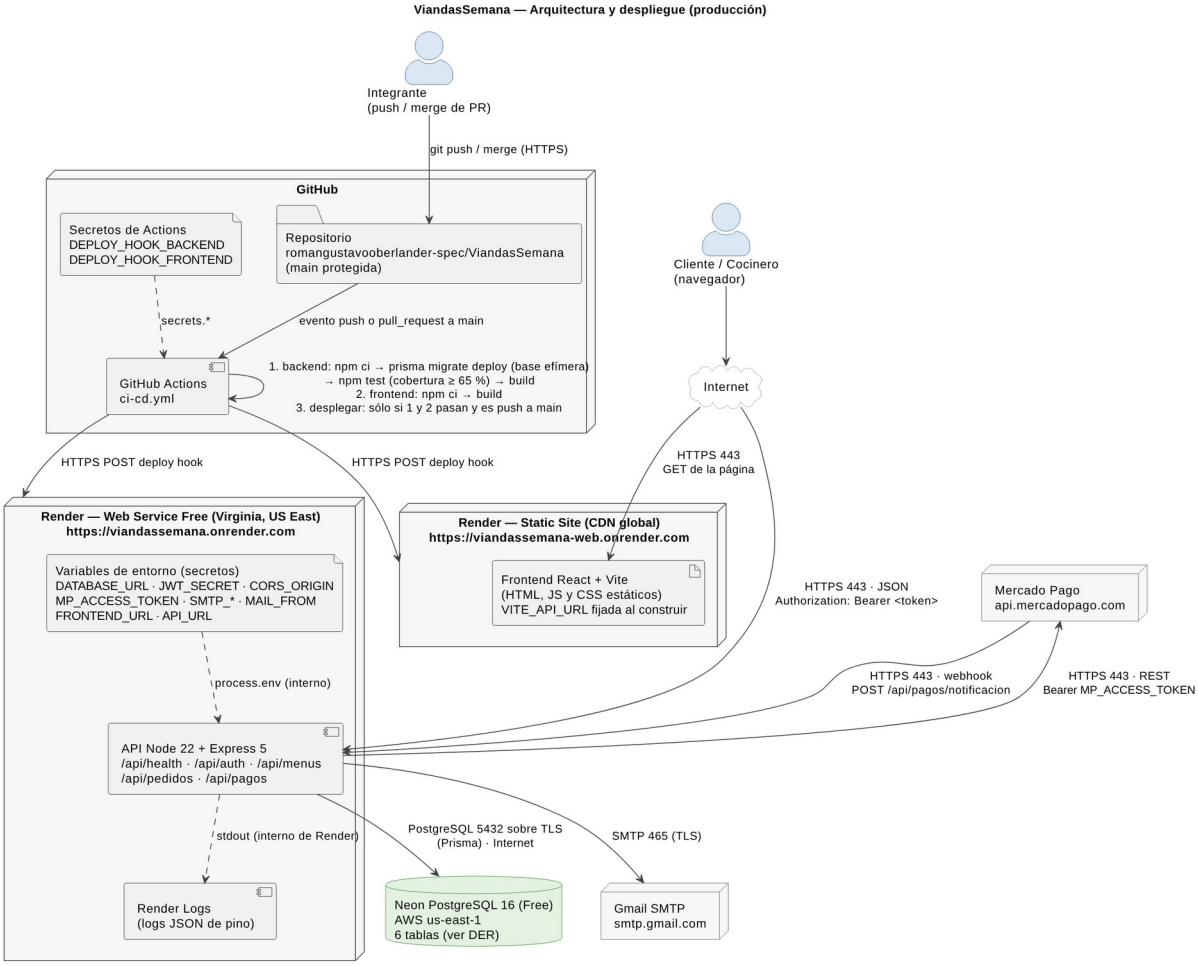
Pedido

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
cantidad	INTEGER	NOT NULL	Viandas reservadas (≥ 1)
estado	EstadoPedido	NOT NULL, DEFAULT PENDIENTE_PAGO	PENDIENTE_PAGO, CONFIRMADO o CANCELADO
usuarioId	INTEGER	NOT NULL, FK → Usuario.id	Cliente que reservó
menuId	INTEGER	NOT NULL, FK → MenuDia.id	
creadoEn	TIMESTAMP(3)	NOT NULL, DEFAULT now	

Pago

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PK	
pedidoId	INTEGER	NOT NULL, UNIQUE, FK → Pedido.id	
monto	INTEGER	NOT NULL	Pesos: cantidad × precio
estado	EstadoPago	NOT NULL, DEFAULT PENDIENTE	PENDIENTE, APROBADO, RECHAZADO, CANCELADO o DEVUELTO
preferenciaId	TEXT	NULL, UNIQUE	Preferencia de Checkout Pro
urlPago	TEXT	NULL	init_point de la preferencia
mpPagoId	TEXT	NULL	Id del pago en Mercado Pago
venceEn	TIMESTAMP(3)	NOT NULL	Hasta cuándo se puede pagar
creadoEn	TIMESTAMP(3)	NOT NULL, DEFAULT now	
actualizadoEn	TIMESTAMP(3)	NOT NULL	Lo actualiza Prisma en cada cambio

4. Arquitectura y despliegue



Archivo editable: diagramas/arquitectura.drawio.

4.1 Componentes

Componente	Tecnología	Plataforma
Frontend	React 18 + Vite, sitio estático	Render Static Site — https://viandassemana-web.onrender.com
API	Node 22 + Express 5 + TypeScript + Prisma	Render Web Service, plan Free — https://viandassemana.onrender.com
Base de datos	PostgreSQL 16	Neon, plan Free (privada)
Pagos	Mercado Pago Checkout Pro	servicio externo
Correo	SMTP de Gmail (cuenta del proyecto)	servicio externo
Logs	pino → Render Logs	Render
Repositorio y CI/CD	GitHub + GitHub Actions	GitHub

El plan gratuito de Render duerme la API tras 15 minutos sin tráfico; la primera petición posterior tarda alrededor de un minuto.

4.2 Comunicaciones

Desde → hacia	Protocolo y puerto
Navegador → frontend	HTTPS, 443
Navegador → API	HTTPS, 443, JSON, Authorization: Bearer <token>
API → Neon PostgreSQL	PostgreSQL sobre TLS, 5432 (Prisma)
API → Mercado Pago	HTTPS, 443, REST con Bearer MP_ACCESS_TOKEN
Mercado Pago → API (webhook)	HTTPS, 443, POST /api/pagos/notificacion
API → Gmail	SMTP con TLS, 465
GitHub Actions → Render	HTTPS, 443, POST a los deploy hooks

El frontend nunca habla con la base de datos: todo pasa por la API.

4.3 Secretos

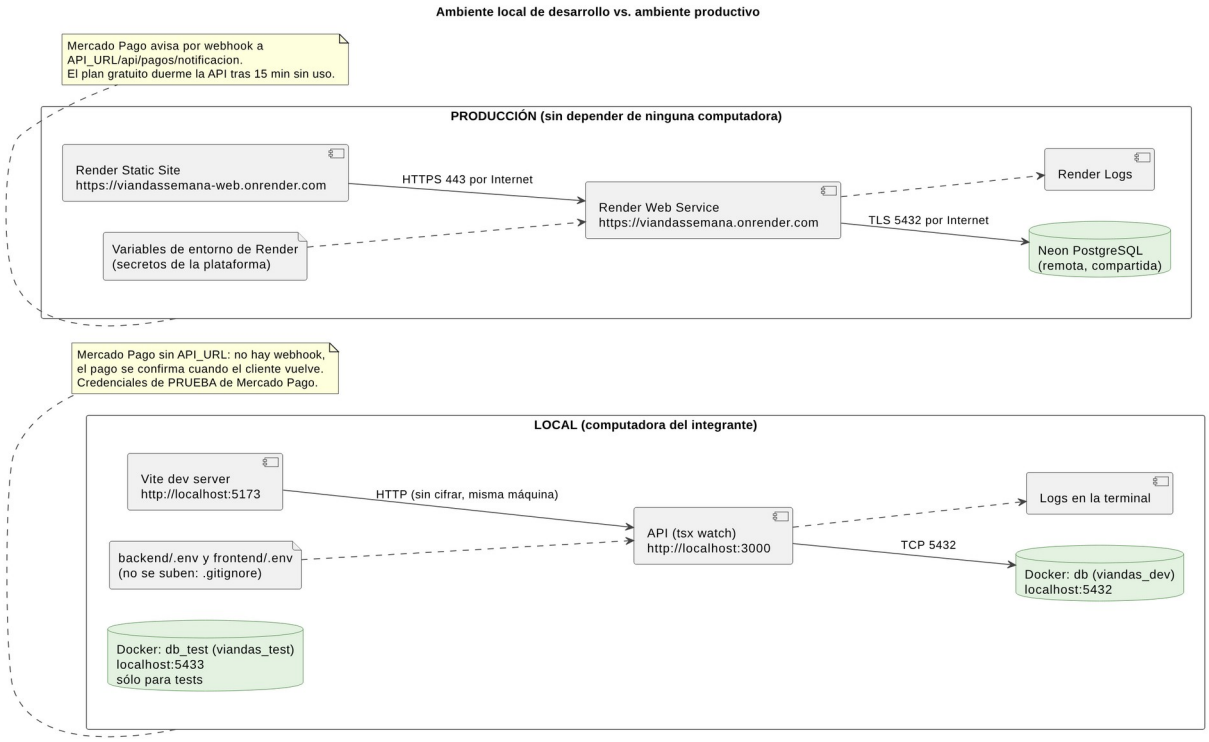
Secreto	Dónde se carga	Cómo llega
DATABASE_URL, JWT_SECRET, MP_ACCESS_TOKEN, SMTP_USER, SMTP_PASS	Variables de entorno del Web Service en Render	process.env → config/env.ts (única lectura)
VITE_API_URL	Variable del Static Site en Render	Vite la escribe al construir (es pública)
DEPLOY_HOOK_BACKEND, DEPLOY_HOOK_FRONTEND	Secretos de GitHub Actions	${{ secrets.* }} en el job desplegar

Ningún secreto está en el repositorio: `.env` está en `.gitignore` y `.env.example` sólo tiene nombres.

4.4 Del push a producción

1. Un integrante sube una rama y abre un pull request a main (rama protegida).
2. GitHub Actions corre los jobs backend (migraciones, 78 tests, cobertura ≥ 65 %, build) y frontend (build).
3. Se hace merge; en el push a main se vuelve a correr el pipeline.
4. Sólo si los dos jobs pasaron, el job desplegar llama a los deploy hooks de Render.
5. Render descarga el commit de main, lo construye y reemplaza la versión en línea.

4.5 Ambiente local y de producción



Archivo editable: diagramas/ambientes.drawio.

Aspecto	Local	Producción
Frontend	npm run dev, http://localhost:5173	Render Static Site, HTTPS
API	npm run dev, http://localhost:3000	Render Web Service, HTTPS
Base de datos	viandas_dev en localhost:5432	Neon PostgreSQL remota
Base de tests	viandas_test en localhost:5433	contenedor efímero en GitHub Actions
Configuración	archivos .env (no se suben)	variables de Render y secretos de GitHub
Mercado Pago	credenciales de prueba; el pago se confirma cuando el cliente vuelve	webhook a la API pública
Logs	en la terminal	Render Logs

La diferencia entre ambientes se resuelve sólo por configuración: el mismo código corre en los dos.

5. Decisiones técnicas

Criterios: **costo** (todo debía ser gratuito), **curva de aprendizaje** para dos integrantes, **calidad de la documentación** y **adecuación al problema** (pocas pantallas, reglas de cupo y horario que se cumplen aun con pedidos simultáneos, y cobro en pesos).

1. Frontend — React 18 + Vite (TypeScript), sin librerías de UI ni router

Alternativas: Angular; Vue 3; Next.js.

Fundamento: React tiene la documentación y la cantidad de ejemplos más grandes y ya lo conocíamos; Vite construye un sitio estático que se publica gratis en un CDN. Angular exige aprender mucho más para cuatro pantallas; Next.js agrega renderizado en el servidor, que no necesitamos.

Se resigna: las herramientas que trae Angular "de fábrica" y el SEO del renderizado en servidor; las pestañas se manejan con estado.

2. Backend — Node 22 + Express 5 + TypeScript, validación con zod

Alternativas: Python con FastAPI; Java con Spring Boot.

Fundamento: el mismo lenguaje que el frontend; Express es mínimo y muy documentado; Express 5 lleva solos los errores de las rutas async al manejador central; zod da errores 400 claros por campo. Spring Boot es más pesado de aprender y de correr en un plan gratuito de 512 MB.

Se resigna: la estructura impuesta de un framework más grande y la documentación automática de la API de FastAPI.

3. Base de datos — PostgreSQL en Neon, con Prisma

Alternativas: MySQL en Aiven o Railway; MongoDB Atlas; Supabase.

Fundamento: el negocio es relacional y la regla de cupo necesita transacciones con bloqueo de fila (SELECT … FOR UPDATE), que PostgreSQL resuelve bien. Neon es gratis, remoto y con TLS. Prisma da migraciones versionadas y consultas tipadas.

Se resigna: el panel y la autenticación incluidos de Supabase y algo de control fino del SQL. El plan gratuito se suspende sin uso.

4. Autenticación — propia, con bcrypt, JWT (8 h) y roles en la tabla Rol

Alternativas: Auth0 o Clerk; sesiones con cookie guardadas en la base.

Fundamento: son pocas reglas, se prueban con tests de integración (401 y 403) y se explican completas en la defensa. Para cerrar sesión de verdad se agregó versionToken.

Se resigna: inicio de sesión con Google y verificación del email; el token en localStorage depende de que el frontend no tenga XSS.

5. Despliegue y CI/CD — Render + GitHub Actions

Alternativas: Vercel + Railway; un VPS con Docker.

Fundamento: Render tiene plan gratuito para los dos componentes, HTTPS automático, secretos y deploy hooks; GitHub Actions permite que el despliegue dependa de los tests y la cobertura (≥ 65 %).

Se resigna: la API se duerme tras 15 minutos sin uso y los logs gratuitos guardan poco historial.

6. Tecnología avanzada — integración con Mercado Pago, más logs estructurados

Alternativas: observabilidad sola con una herramienta externa; caché con Redis.

Fundamento: resuelve un problema real: las reservas sin pagar ocupaban cupo. Ahora una reserva sólo se confirma con el pago aprobado y se libera sola si vence. El estado se consulta siempre a Mercado Pago y la falla o demora del servicio responde 502/503 sin dejar reservas trabadas.

Se resigna: Mercado Pago cobra comisión por venta; en local no llega el webhook; sin herramienta de logs externa el historial es corto.

Declaración de uso de inteligencia artificial

Durante el desarrollo se usó **Claude (Anthropic)**, a través de Claude Code, para: generación de código del backend y del frontend, escritura de tests, interpretación de errores del pipeline y de Mercado Pago, redacción de la documentación y elaboración de los diagramas (DFD, casos de uso, DER y arquitectura, a partir del código del repositorio). Los integrantes definieron el producto y sus reglas de negocio, eligieron entre las alternativas de esta sección, probaron el sistema funcionando y asumen la responsabilidad de comprender y explicar todo lo entregado.

Revisar y completar esta declaración con lo que efectivamente hizo cada integrante.